

Documento de Refactorización

Proyecto: Aseguradora VidaFutura

Materia: Programación Orientada a Objetos

Alumno: Mario Alexander Aguirre Lopez

Resumen Ejecutivo: Este documento presenta los cambios realizados durante la **Fase 4 (Refactorización)** y **Fase 5 (Migración a MySQL)** del proyecto. El objetivo fue eliminar código duplicado, unificar la interfaz de usuario y migrar la persistencia de archivos CSV a una base de datos MySQL relacional.

1. CRUD — Antes vs Después

X ANTES Código duplicado por entidad (~25 funciones)

Cada una de las 5 entidades (Cliente, Póliza, Beneficiario, Pagos, Siniestros) tenía **4 funciones CRUD idénticas** escritas a mano:

```

# Había que escribir esto 5 VECES (una por entidad):

def agregar_cliente():
    os = Cliente()
    os.capturardatos()
    archivo_clientes.agregar(os.devolverdatos())
    print("Guardado con éxito")

def listar_clientes():
    lista = archivo_clientes.leerDatos()
    if not lista:
        print("No hay clientes registrados")
    else:
        for i, item in enumerate(lista):
            # Impresión campo por campo ...

def modificar_cliente():
    listar_clientes()
    num = int(input("¿Qué número quieres modificar?: ")) - 1
    c = Cliente()
    c.capturardatos()
    archivo_clientes.modificar(num, c.devolverdatos())

def borrar_cliente():
    listar_clientes()
    num = int(input("¿Qué número quieres borrar?: ")) - 1
    archivo_clientes.eliminar(num)

```

Total: ~20 funciones casi idénticas repartidas entre los archivos de cada clase.

✓ DESPUÉS **4 funciones genéricas (CRUDGenerico.py)**

```
def Agregar_generico(Clase, archivo, nombre):
    os = Clase()
    os.capturardatos()
    archivo.agregar(os.devolverdatos())
    print(f"{nombre} guardado con éxito")

def Listar_generico(archivo, nombre):
    lista = archivo.leerDatos()
    if not lista:
        print(f"No hay {nombre}s registrados")
        return False
    else:
        enca = archivo.devolverencabezados()
        for i, item in enumerate(lista):
            datos = item.split(",")
            for k, v in zip(enca, datos):
                print(f"{k}: {v}")
        return True
```

Resultado: de ~20 funciones → 4 funciones. Reducción del

80% del código CRUD

2. Submenús — Antes vs Después

X ANTES 5 submenús casi idénticos (~150 líneas)

Se repetía esta estructura 5 veces con diferentes nombres:

```
def menu_Clientes():
    while True:
        print("\n--- Menú Clientes ---")
        print("1. Agregar")
        print("2. Listar")
        print("3. Modificar")
        print("4. Borrar")
        print("5. Regresar")
        opcion = int(input("Elige: "))
        if opcion == 1:
            agregar_cliente()    # función específica
        elif opcion == 2:
            listar_clientes()    # función específica

def menu_Poliza():
    # Exactamente lo mismo pero con Póliza...

def menu_Beneficiarios():
    # Exactamente lo mismo pero con Beneficiario...
```

✓ DESPUÉS 1 menú genérico + 5 llamadas

```
def menu_generico(titulo, Clase, archivo, nombre):
    while True:
        print(f"\n--- Menú {titulo} ---")
        print("1. + Agregar")
        print("2. 📄 Listar")
        print("3. 🔄 Modificar")
        print("4. 🗑 Borrar")
        print("5. ⬅ Regresar")
        # ...

# Cada menú ahora es solo 2 líneas:
def menu_Clientes():
    menu_generico("Clientes", Cliente, archivo_clientes, "Cliente")

def menu_Poliza():
    menu_generico("Pólizas", Poliza, archivo_poliza, "Póliza")
```

Resultado: de ~150 líneas repetidas → ~35 líneas totales. Reducción del 77%

3. Persistencia — Antes vs Después

X ANTES Archivos CSV (ClassArchivos.py)

```
class ArchivosCSV:
    def __init__(self, archivo, encabezado):
        self.archivo = archivo          # Ruta al archivo .csv
        self.encabezado = encabezado

    def agregar(self, dato):
        with open(self.archivo, "a") as a:
            a.write(f"{siguiente},{dato}\\n")

    def leerDatos(self):
        with open(self.archivo, "r") as a:
            # Lee línea por línea ...
```

Problemas:

- Sin integridad referencial (puedes borrar un cliente con pólizas activas)
- Sin concurrencia (archivos bloqueados si se abren en Excel)
- IDs se recalculan al borrar (inconsistencia)

✓ DESPUÉS MySQL (ClassMySQL.py + MySqlConnection.py)

```

class ConexionMySQL:
    def ejecutar(self, sql, parametros=None):
        """INSERT, UPDATE, DELETE con commit/rollback"""
        conex = self.conectar()
        cursor = conex.cursor()
        try:
            cursor.execute(sql, parametros)
            conex.commit()
        except mysql.connector.Error as e:
            conex.rollback()
        finally:
            cursor.close()
            conex.close()

class ArchivosMySql:
    def __init__(self, tabla, encabezado):
        self.tabla = tabla          # Nombre de la tabla MySQL
        self.encabezado = encabezado

    def agregar(self, dato):
        sql = f"INSERT INTO {self.tabla} ({cols}) VALUES ({placeholders})"
        db.ejecutar(sql, tuple(valores))

```

Ventajas obtenidas:

-  Foreign keys con integridad referencial
-  IDs auto-increment (nunca se repiten)
-  Transacciones con rollback ante errores
-  Consultas parametrizadas (prevención de SQL injection)

Clave del diseño: Patrón Estrategia

La migración fue posible porque ArchivosMySql implementa **exactamente los mismos métodos** que ArchivosCSV:

Método	ArchivosCSV	ArchivosMySQL
agregar(dato)	Escribe en archivo	INSERT INTO
leerDatos()	Lee línea por línea	SELECT *
modificar(id, dato)	Reescribe archivo	UPDATE WHERE
eliminar(indice)	Pop + reescribir	DELETE WHERE
devolverencabezados()	Split del encabezado	Split del encabezado

Para migrar todo el sistema solo se cambió 1 línea en SubMenus.py:

```
- from ClassArchivos import archivo_clientes, archivo_poliza, ...
+ from ClassMySQL import archivo_clientes, archivo_poliza, ...
```

4. Validaciones de Fechas — Antes vs Después

X ANTES Fechas guardadas como objetos datetime

```
# En devolverdatos():
return f"{self.fecha_inicio}, {self.fecha_fin}, ..."
# Resultado en BD: "2026-01-15 00:00:00" ← MySQL rechaza esto
```

✓ DESPUÉS Formato explícito YYYY-MM-DD

```
# En devolverdatos():
return f"{self.fecha_inicio.strftime('%Y-%m-%d')}, {self.fecha_fin.strftime('%Y-%m-%d')}"
# Resultado en BD: "2026-01-15" ← Compatible con DATE de MySQL
```

Aplicado en: ClassPoliza.py, ClassSiniestros.py, ClassPagos.py

5. Resumen de Métricas

Métrica	Antes	Después	Mejora
Funciones CRUD	~20	4	-80%
Líneas en SubMenus.py	~150	63	-58%
Persistencia	CSV (archivos planos)	MySQL (relacional)	↑ Profesional
Integridad de datos	Ninguna	Foreign Keys	↑ Robustez
Líneas para migrar	—	1 línea de import	Patrón Estrategia
Tests automatizados	0	20/20	↑ Confiabilidad